

Enterprise AI Operations Copilot

RAG-Based Root Cause Analysis for Enterprise Operations —
Multi-Agent AI · LangGraph · Qdrant Vector Search · Gemini · Langfuse

Document	Technical Project Report
Project	Enterprise AI Operations Copilot
Version	1.1.0 (backend/frontend app version; git tag v1.0.0)
Report Date	September 4, 2026
Backend Stack	Python 3.12 · FastAPI · LangGraph · Qdrant · Gemini 2.5 Flash
Frontend Stack	React 19 · Vite 6 · Plain CSS · react-markdown
Status	Functional prototype — local deployment, 26/26 tests passing

Table of Contents

1. Executive Summary

2. Project Overview

2.1 Purpose and Problem Statement

2.2 Objectives

2.3 Scope

2.4 Target Users and Use Cases

3. Functional Requirements

4. Non-Functional Requirements

5. Features & Functionalities

5.2 Partially Implemented / Recommended

6. Technology Stack

7. System Architecture

7.1 Architecture Explanation

8. Detailed Component Architecture

9. Frontend Architecture

10. Backend Architecture

11. API Documentation

12. Data Architecture

13. Authentication & Authorization

14. AI / LLM / RAG / MCP

14.1 RAG Pipeline

14.2 Multi-Agent Design

14.3 MCP

14.4 Prompt Engineering Notes

15. Complete Application Workflow

16. Complete Data Flow

17. Project Folder Structure

18. Module & Implementation Details

19. Validation & Error Handling

20. Security Analysis

20.1 Implemented Security

20.2 Recommended Security Improvements

21. Testing

4

4

4

4

4

5

5

6

6

6

7

7

8

9

9

10

11

11

12

12

12

13

13

13

13

14

14

14

15

15

16

17

17

17

17

- 22. Verification & Quality Assurance 18
- 23. Performance Analysis 18
- 24. Scalability 19
- 25. Deployment & DevOps 19
- 26. Installation & Setup 20
- 27. Problems Found & Improvements Made 20
- 28. Current Strengths 21
- 29. Known Limitations 22
- 30. Recommended Improvements 22
 - 30.1 High Priority 22
 - 30.2 Medium Priority 22
 - 30.3 Low Priority 22
- 31. Future Enhancements 23
- 32. Maintenance & Support 23
- 33. GitHub / Repository Readiness 23
- 34. Resume / Portfolio Value 24
- 35. Final Technical Summary 24
- 36. Appendix 24
 - A. API Summary 25
 - B. Data Summary 25
 - C. Environment Variable Names (values never documented) 25
 - D. Key Configuration Defaults 25
 - E. Glossary 25
 - F. Verification Information 25

1. Executive Summary

The **Enterprise AI Operations Copilot** is an AI-powered application that performs automated Root Cause Analysis (RCA) of enterprise incidents. A user asks a natural-language question such as *"Why is payroll failing?"* and the system retrieves relevant incidents (semantic vector search), correlates them with deployments, prioritises relevant log entries, retrieves matching operational runbooks, validates the aggregated evidence, and generates a grounded, structured RCA report using Google Gemini 2.5 Flash.

The system is deliberately engineered to be **honest and grounded**: the LLM never reports a confidence higher than the evidence allows, timelines are built only from real timestamps, runbook content is presented as guidance (never as observed evidence), and if the LLM is unavailable the API still returns the observed evidence with an explicit notice. This anti-hallucination posture is enforced in code and covered by unit tests.

Major capabilities (all verified in the codebase):

- Multi-stage LangGraph investigation workflow (8 deterministic nodes) with per-stage tracing and real-time SSE progress streaming to the UI.
- RAG over incidents: SentenceTransformer (BAAI/bge-small-en-v1.5, 384-dim) embeddings + embedded Qdrant vector store with payload indexes and a relevance-score threshold.
- Deployment correlation via explicit deployment_id, with a documented service-name fallback.
- Structured evidence layer (Pydantic models) separating OBSERVED EVIDENCE, RUNBOOK GUIDANCE and AI ANALYSIS in the UI.
- Grounded Gemini prompt with retries, timeout, and an evidence-only fallback report.
- Per-IP sliding-window rate limiting, strict CORS, structured error envelopes, and fail-soft Langfuse observability.
- React 19 chat-style UI with dark/light theme, investigation history, evidence accordions, timeline, and feedback capture.
- 26/26 pytest tests passing (external services mocked - no API credits consumed); production frontend build verified.

Current implementation status: functional single-user prototype running locally. Authentication, multi-user persistence (database), Docker/CI-CD, and production hosting are not implemented and are documented in the README as future evolution.

2. Project Overview

2.1 Purpose and Problem Statement

During production incidents, operations engineers must manually correlate tickets, deployment records, log files and runbooks across scattered systems. This is slow, error-prone, and often produces speculative conclusions. The project automates the evidence-gathering and correlation work and produces a first-draft RCA whose every claim is traceable to retrieved evidence.

2.2 Objectives

- Automate incident - deployment - log - runbook correlation for any natural-language question.
- Ground the LLM strictly in retrieved evidence; never invent incidents, timestamps or root causes.
- Make confidence transparent: evidence strength is computed by a deterministic validator, independent of the LLM.
- Show real pipeline progress (SSE) rather than fake progress indicators.
- Keep observability optional and fail-soft so the pipeline works without external dependencies.

2.3 Scope

In scope: investigation of incidents recorded in the bundled sample data set (20 incidents, 20 deployments, 550 log lines, 5 markdown runbooks); semantic incident retrieval; structured RCA generation; SSE streaming; local

persistence of history/feedback in the browser.

Out of scope (not identified in the current implementation): live ingestion from ticketing/APM systems, user accounts/authentication, server-side persistence, multi-tenant support, and production deployment infrastructure.

2.4 Target Users and Use Cases

Aspect	Detail
Primary user	Operations / SRE engineers investigating incidents
Secondary users	Engineering managers reviewing RCA quality; support teams correlating customer issues
Use case 1	"Why is payroll failing?" - full RCA with incident, deployment, log and runbook evidence
Use case 2	"Was the recent deployment responsible for the outage?" - deployment correlation
Use case 3	"Analyze authentication failures" - JWT/auth runbook guidance + related incidents
Expected outcome	Faster triage with a structured, evidence-grounded RCA and an inspectable evidence bundle

3. Functional Requirements

The requirements below are derived directly from the implemented code (routes, workflow nodes, and UI components).

ID	Requirement	Implementation	Status
FR-1	Accept a natural-language investigation question	POST /investigate and /investigate/stream; Pydantic InvestigateRequest (8-500 chars, non-blank)	Implemented
FR-2	Stream real workflow progress	SSE events per LangGraph node completion (NODE_STAGE_IDS); LoadingState renders only backend-confirmed stages	Implemented
FR-3	Semantic incident retrieval with relevance filtering	Qdrant cosine search; scores below MIN_RELEVANCE_SCORE (0.30) dropped in incident_agent	Implemented
FR-4	Correlate incidents with deployments	deployment_agent: primary by deployment_id; fallback by service only when no ID match	Implemented
FR-5	Prioritise ERROR/WARN log lines; cap context size	log_agent regex parser, severity ranking, MAX_LOG_EVIDENCE = 40	Implemented
FR-6	Retrieve matching runbook as guidance, not evidence	runbook_agent keyword scoring over 5 curated markdown runbooks	Implemented
FR-7	Validate evidence and assess strength	evidence_validation node: deterministic HIGH/MEDIUM/LOW/INSUFFICIENT heuristic	Implemented
FR-8	Generate grounded RCA via LLM	gemini_rca_agent structured prompt; markdown parsed into InvestigationResult; confidence clamped by evidence layer	Implemented
FR-9	Graceful degradation without LLM	_fallback_result: evidence-only report with explanatory message (verified live)	Implemented
FR-10	Expose inspectable evidence in the UI	EvidencePanel accordions with source tags (OBSERVED EVIDENCE / RUNBOOK GUIDANCE / AI ANALYSIS)	Implemented
FR-11	Persist investigation history and feedback	localStorage (copilot-history max 20, copilot-feedback max 100); no backend persistence	Implemented (client-side)
FR-12	Semantic search endpoint	POST /search: query (2-300 chars), limit (1-20) over the Qdrant collection	Implemented
FR-13	Health monitoring	GET /health returns status, qdrant, gemini_configured; UI polls every 30 s	Implemented
FR-14	User accounts, login, server-side history	-	Not identified in the current implementation

4. Non-Functional Requirements

Category	Observed in Implementation	Assessment
Performance	Data files cached via lru_cache; embedding model loaded once at import; SSE streams stage-by-stage; 180 s client timeout for LLM latency; stage timings logged in ms	Good for prototype scale
Scalability	Embedded Qdrant and in-memory rate limiter are single-process only; README documents Qdrant server mode for multi-instance	Limited - see Section 24
Security	Strict CORS allow-list (no wildcard); rate limiting on expensive endpoints; generic error envelopes; no secrets in code; input length validation mirrored front/back	Solid basics; no authentication yet
Reliability	Defensive nodes record failures and continue; Gemini retry x3 + fallback; Langfuse fail-soft; Qdrant collection auto-created on startup if missing	Strong failure isolation
Maintainability	Single config module; typed Pydantic contracts between stages; docstrings; modular agents; 26 unit tests	High
Usability	Chat UI, example prompts, live stage progress, dark/light theme, evidence accordions, searchable history	High
Availability	Single local uvicorn process; health endpoint exists but no orchestrator/restart policy	Not addressed - recommendation
Accessibility	aria-labels/roles on key controls (input, toggles, status pill, aria-live loading)	Partially addressed
Data integrity	Deterministic UUIDv5 point IDs make seeding idempotent; timeline built only from real timestamps; seed validates required fields	Good within file-based scope

5. Features & Functionalities

RCA Investigation Pipeline

What: Turns a question into a structured RCA (summary, impact, root cause, confidence, contributing factors, remediation, preventive actions, timeline, evidence bundle, validation verdict).

How: LangGraph StateGraph with 8 sequential nodes; each node wraps an agent and records errors defensively (workflows/rca_graph.py).

Semantic Incident Search (RAG)

What: Finds incidents semantically similar to the question.

How: SentenceTransformer embeds the query; Qdrant cosine top-k with payload indexes; threshold filtering (vector_store/, agents/incident_agent.py).

Deployment Correlation

What: Links matched incidents to the exact deployments that may have caused them.

How: Explicit deployment_id join; service-name fallback only when no ID matches (agents/deployment_agent.py).

Log Evidence Extraction

What: Surfaces relevant ERROR/WARN lines without dumping raw logs into the prompt.

How: Regex parser for 'timestamp service LEVEL message' lines; relevance scoring; cap of 40 lines (agents/log_agent.py).

Runbook Guidance

What: Provides operational remediation context distinct from observed evidence.

How: Keyword-scored selection over 5 curated markdown runbooks, capped at 4000 chars (agents/runbook_agent.py).

Evidence Validation & Confidence Gating

What: Computes evidence strength and clamps the LLM's confidence claim.

How: Deterministic heuristic using incident relevance (0.45 threshold for HIGH) and corroboration counts (rca_graph.py, _final_confidence).

Grounded LLM RCA Generation

What: Generates the human-readable RCA report.

How: Structured prompt with explicit anti-hallucination instructions; markdown parsed into structured sections; retries; evidence-only fallback (agents/gemini_rca_agent.py).

Real-Time Progress Streaming

What: Users see each pipeline stage complete as it happens.

How: POST /investigate/stream emits SSE events driven by LangGraph stream_mode='updates'; LoadingState marks stages done only on backend confirmation.

Evidence Inspection UI

What: Every RCA is accompanied by its raw evidence.

How: EvidencePanel accordions: incidents (with relevance labels), deployments, copyable log lines, runbook card, AI-cited facts.

History & Feedback

What: Recent investigations are searchable and restorable; thumbs-up/down feedback with reason capture.

How: localStorage keys copilot-history (max 20) / copilot-feedback (max 100); Sidebar search; Feedback component.

Health Indicator

What: Shows backend connectivity at a glance.

How: Header polls GET /health every 30 s; status pill shows Systems operational / Backend offline.

Theming

What: Dark (default) and light themes.

How: document.documentElement.dataset.theme toggle persisted in localStorage (copilot-theme).

5.2 Partially Implemented / Recommended

- **MCP tool server (partially implemented):** backend/mcp_server/server.py exposes four real tools (search_incidents_tool, get_deployment, search_logs, get_runbook) via FastMCP, but it is an isolated optional integration - no agent currently consumes it and it is not wired into the main API.
- **Langfuse tracing (implemented, optional):** one trace per investigation with per-stage spans and stage timings; fully fail-soft when unconfigured.
- **Not implemented (recommended):** authentication/authorization, server-side persistence, Docker/CI-CD, automated E2E tests, live telemetry ingestion.

6. Technology Stack

Technology	Version	Purpose	Where Used
Python	3.12 (3.12.10 verified)	Backend language	backend/
FastAPI	0.115.6	HTTP API framework, CORS, validation, SSE	backend/app.py, api/routes.py
Pydantic	2.10.4	Request/response models and evidence contracts	api/routes.py, models/evidence.py

Technology	Version	Purpose	Where Used
uvicorn[standard]	0.34.0	ASGI server	backend launch
LangGraph	0.2.60	Investigation workflow orchestration	workflows/rca_graph.py
google-genai	0.6.0	Gemini LLM client	agents/gemini_rca_agent.py
Gemini 2.5 Flash	gemini-2.5-flash (default)	RCA generation LLM (external API)	configurable via GEMINI_MODEL
SentenceTransformers	3.3.1	Embeddings: BAAI/bge-small-en-v1.5 (384-dim)	vector_store/search_service.py, scripts/seed.py
Qdrant client	1.12.1	Vector database (embedded local mode)	vector_store/
Langfuse SDK	>=3.0.0,<4	Optional LLM observability (traces/spans)	observability/
python-dotenv	1.0.1	Environment configuration	config.py, observability/langfuse_client.py
pytest	8.3.4	Backend testing	backend/tests/
httpx	0.28.1	Test client transport	tests via FastAPI TestClient
FastMCP (mcp)	package 'mcp'	Optional MCP tool server	backend/mcp_server/server.py
React	^19.2.6	Frontend UI framework	frontend/src/
Vite	^6.0.5 (6.4.3 verified)	Dev server and production build	frontend/vite.config.js
react-markdown	^9.0.1	Renders markdown RCA sections	RCAReport.jsx, InvestigationMessage.jsx
ESLint	^9.17.0	Frontend linting (npm run lint)	eslint.config.js
JavaScript (ESM)	ES2022+ (Node 24 verified)	Frontend language	frontend/src/
CSS (plain)	-	Theming and responsive layout	App.css, index.css

Not identified in the current implementation: relational/document databases (PostgreSQL, Supabase, MongoDB), authentication libraries (JWT/OAuth), Docker, CI/CD tooling, cloud SDKs, E2E/UI testing frameworks (Playwright, Cypress), and monitoring infrastructure.

7. System Architecture

The application follows a classic two-tier client/server design with an internal multi-agent pipeline. The React SPA communicates only with the FastAPI backend; all retrieval, correlation, validation and LLM work happens server-side behind a single API surface.

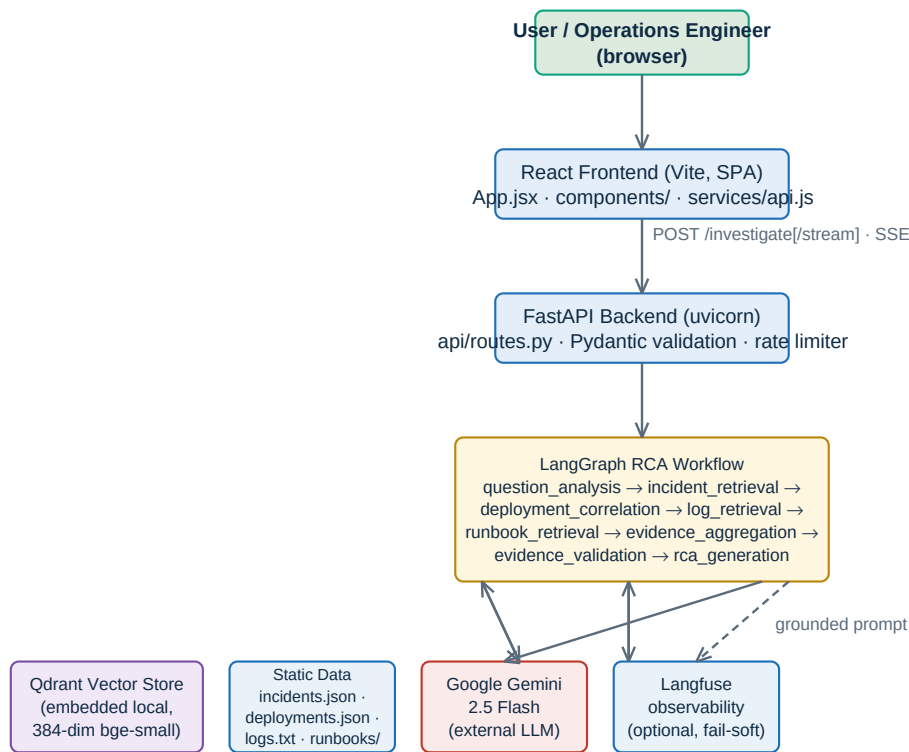


Figure 1 - System architecture: frontend, API, LangGraph pipeline and backing stores

7.1 Architecture Explanation

- **Client tier:** React 19 SPA (Vite). All backend access goes through services/api.js using VITE_API_URL - no hardcoded URLs in components. Streaming progress arrives via SSE; history/theme live in localStorage.
- **API tier:** FastAPI exposes /health, /search, /investigate and /investigate/stream. Pydantic models validate input; a per-IP rate limiter guards the expensive endpoints; all errors use a uniform {status, message} envelope.
- **Workflow tier:** a compiled LangGraph StateGraph orchestrates 8 nodes. Each node catches its own exceptions and appends to state['errors'], so partial evidence still produces a report.
- **Data tier:** an embedded Qdrant instance persists incident vectors on disk (backend/qdrant_data); deployments/logs/runbooks are static files cached in memory via lru_cache.
- **External services:** Google Gemini (LLM) and Langfuse Cloud (optional tracing) are the only external dependencies; both calls fail soft or fall back.
- **Deployment shape today:** everything runs on one machine - uvicorn process, embedded Qdrant, Vite dev server (or built dist/).

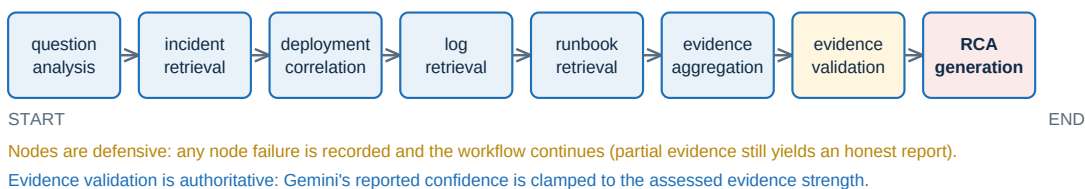


Figure 2 - LangGraph RCA workflow node sequence (start to END)

8. Detailed Component Architecture

Component	Responsibility	Inputs	Outputs	Key Interactions / Files
FastAPI app	App lifecycle, CORS, error handlers, startup validation of Qdrant collection	HTTP requests	JSON / SSE responses	app.py, config.py, vector_store/qdrant_setup.py
Routes	Request validation, rate limiting, tracing, response envelope	question / query JSON	InvestigationResponse, SearchResponse, HealthResponse, SSE events	api/routes.py, models/evidence.py, utils/rate_limit.py
LangGraph workflow	Orchestrate the 8-node investigation; defensive error capture; timeline; validation	RCASState(question)	RCASState(investigation)	workflows/rca_graph.py
Incident agent	Semantic retrieval + relevance threshold filtering	question, limit=5	list[IncidentEvidence] + scores	vector_store/search_service.py
Deployment agent	deployment_id correlation with service fallback	list[IncidentEvidence]	list[DeploymentEvidence]	utils/data_store.get_deployments
Log agent	Parse, score, prioritise and cap log evidence	incidents, keywords, deployments	list[LogEvidence] (max 40)	utils/data_store.get_logs
Runbook agent	Best-matching operational runbook	question, incidents	RunbookEvidence None	data/runbooks/*.md
Evidence validator	Strength heuristic + notes + timeline (real timestamps only)	EvidenceBundle	EvidenceValidation, list[TimelineEvent]	workflows/rca_graph.py
Gemini RCA agent	Grounded prompt, retries, markdown parse, confidence clamping, fallback	EvidenceBundle, EvidenceValidation, timeline	InvestigationResult (never raises)	google-genai client, observability.tracing
Qdrant layer	Collection lifecycle, payload indexes, vector search	query vector, optional filters	scored points	vector_store/qdrant_setup.py, search_service.py
Data store	Cached loading of static enterprise data	data/ files	deployments list, log lines, runbooks dict	utils/data_store.py (lru_cache)
Tracer	One Langfuse trace per investigation; stage spans + timings; noop when unconfigured	question length	trace metadata, ms timings	observability/tracing.py, langfuse_client.py
Rate limiter	Sliding-window per-IP limiting of /investigate	Request	bool allow/deny	utils/rate_limit.py
React App	Layout, message state, SSE handling, history, theme	user question	messages, history, UI	App.jsx, components/*
api.js client	Typed fetch wrappers with timeouts and user-safe errors	question / query	investigation payload / events / health	frontend/src/services/api.js

9. Frontend Architecture

The frontend is a single-page React application with no router - the whole product is one screen (chat-style investigations) plus a collapsible sidebar. State is local React state (useState/useCallback) plus localStorage; no state-management library is used, which is appropriate for the app's size.

Aspect	Implementation
Entry	main.jsx - App.jsx (app-shell: Sidebar + main area)
Screens	Welcome screen with 4 example investigation cards; chat window with user/assistant messages; loading stage list; empty and error states
Components	Header (health pill, theme, sidebar toggle), Sidebar (history search/restoration), InvestigationInput (textarea, char limit, Enter/Shift+Enter), InvestigationMessage, RCAReport (structured sections), EvidencePanel (accordions), Timeline, LoadingState (stage checklist), Feedback
Reusable pieces	SectionCard / ConfidenceBadge / ListSection (RCAReport.jsx); Accordion (EvidencePanel.jsx)
State management	Local useState: messages[], loading, completedStages (Set of backend-confirmed stage ids), history, theme; loadingRef guards concurrent submissions
Routing	None - single view; sidebar navigation and New Investigation reset messages
API communication	services/api.js: investigate() (POST, 180 s timeout), investigateStream() (SSE parser over fetch ReadableStream), checkHealth() (5 s timeout, 30 s poll)
Forms & validation	Question limited to 500 chars (mirrors backend), non-blank submit guard, live counter with limit-hit styling
Auth UI	None - not identified in the current implementation
Error handling	userSafeError() surfaces backend 'message' only; timeout/network errors map to friendly text; error card with role=status notice banner
Responsive design	Sidebar overlay with backdrop on small screens; CSS variables theme both dark/light
UX details	Smooth auto-scroll to latest message; copy-to-clipboard for log evidence; aria-expanded accordions; honest loading copy (no fake checkmarks)

10. Backend Architecture

Aspect	Implementation
Framework	FastAPI 0.115.6 served by uvicorn (single process)
Structure	app.py (app + lifespan + handlers) - api/routes.py (4 endpoints) - workflows/ and agents/ (business logic) - vector_store/ + utils/ (data)
Routes	GET /health, POST /search, POST /investigate, POST /investigate/stream (SSE StreamingResponse with no-cache and X-Accel-Buffering: no)
Middleware	CORS with configurable allow-list (CORS_ORIGINS); GET/POST methods only; Authorization/Content-Type headers only
Validation	Pydantic field constraints + custom validators (blank rejection); mirrored QUESTION_MIN/MAX_LENGTH limits
Business logic	LangGraph nodes calling agent functions; all cross-stage data passes through Pydantic models (models/evidence.py) - no ad-hoc dicts
Authentication	None - endpoints are public on the local network (README requires auth before public deployment)
Authorization	None - not identified in the current implementation
Error handling	Dedicated handlers for HTTPException, RequestValidationError and generic Exception; internals logged, users get generic messages
Data interaction	Qdrant embedded client (path-based); lru_cache for static files; refresh_data_cache() provided for maintenance
External integrations	Gemini generate_content (timeout GEMINI_TIMEOUT_MS, retries GEMINI_MAX_RETRIES); Langfuse traces (fail-soft)
Startup (lifespan)	Warms data cache, logs counts, creates Qdrant collection if missing (warns to run the seed script)

11. API Documentation

Meth od	Endpoint	Purpose	Auth	Request	Response	Errors
GET	/health	Liveness + dependency status	None	-	{status, qdrant: bool, gemini_configured: bool}	500 generic
POST	/search	Semantic incident search	None	{query: 2-300 chars, limit: 1-20 (default 5)}	{status, results: [payloads]}	422; 500 generic
POST	/investigate	Full synchronous RCA	None	{question: 8-500 chars, non-blank}	{status, investigation, report, message}	422; 429 rate limit; 500 generic
POST	/investigate/stream	SSE RCA with live stage events	None	{question: 8-500 chars}	SSE: investigation_started, one _completed per node, investigation_completed (full payload) or investigation_error	429; SSE investigati on_error

The **investigation** object (InvestigationResult) contains: question, status, summary, impact, root_cause, confidence, evidence_strength, contributing_factors, recommended_remediation, preventive_actions, observed_evidence, investigation_details, timeline[], evidence (EvidenceBundle) and validation (EvidenceValidation). report is the same RCA as markdown for fallback rendering. Rate limiting: 10 requests / 60 s per IP by default (configurable). Interactive docs are auto-generated at /docs (Swagger). Every error response uses the same envelope: {"status": "error", "message": "..."}.

12. Data Architecture

There is no relational database. Data lives in three shapes, all local: a vector collection, flat files, and browser localStorage.

Store	Shape	Key fields	Relationships / Constraints
Qdrant collection 'enterprise_incidents'	384-dim vectors + JSON payload, cosine distance	incident_id, deployment_id, service, severity, status, title, root_cause	Payload keyword indexes on service/severity/status; deterministic UUIDv5 point ids (idempotent re-seeding); record count verified after seeding
data/incidents.json (20 records)	JSON array	id, service, severity, status, title, root_cause, deployment_id	Seed script fails on missing required fields; deployment_id references deployments.json
data/deployments.json (20 records)	JSON array	deployment_id, service, time, change	Joined to incidents via deployment_id
data/logs.txt (550 lines)	Plain text lines	timestamp, service, LEVEL, message	Parsed by regex; unparseable lines preserved verbatim (never fabricated fields)
data/runbooks/*.md (5 files)	Markdown	title heading + procedural content	Mapped to domains in runbook_agent.RUNBOOK_KEYWORDS
Browser localStorage	JSON strings	copilot-history (max 20), copilot-feedback (max 100), copilot-theme	Quota-exceeded fallback stores metadata only

Relationships: Incident (1) - deployment_id - (1) Deployment; log lines relate to services mentioned in incidents/deployments; runbooks relate to question/incident domain keywords. **A traditional ER diagram is not applicable** because the stores are a vector collection plus flat files; primary keys are incident.id (and its derived Qdrant point id) and deployment.deployment_id. There are no SQL indexes, foreign keys or row-level security policies - the only indexes are the Qdrant payload indexes listed above.

13. Authentication & Authorization

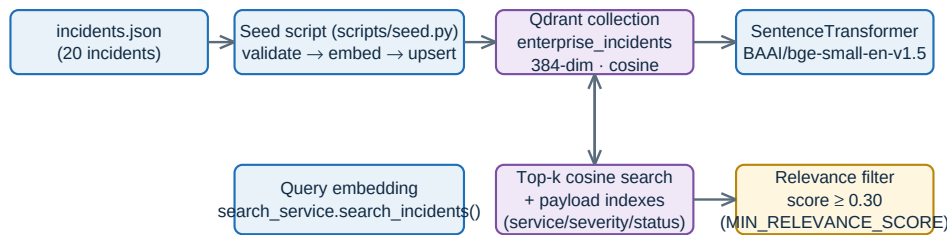
Not identified in the current implementation. The API has no registration, login, sessions, tokens, password handling, roles, permissions or protected routes. This is a deliberate, documented decision for the prototype: the README explicitly states that authentication must be added before any public deployment and that rate limiting

alone does not prevent abuse. The CORS middleware does allow the Authorization header, showing the design anticipates adding token-based auth later. Authorization is therefore covered in the Recommended Improvements (Section 30).

14. AI / LLM / RAG / MCP

AI is the core of the product. The implementation is a disciplined RAG + multi-agent pipeline rather than a raw chat wrapper.

14.1 RAG Pipeline



Chunking: one vector per incident (service/title/root-cause text); runbooks and logs are retrieved by keyword heuristics, not embeddings.

Figure 3 - RAG indexing and retrieval flow for incident evidence

Aspect	Implementation
LLM provider / model	Google Gemini via google-genai SDK; model gemini-2.5-flash (default, configurable); timeout 60000 ms; 2 retries (3 attempts total)
Embedding model	SentenceTransformers BAAI/bge-small-en-v1.5, 384 dimensions, loaded once at module import
Vector database	Qdrant embedded local mode at backend/qdrant_data (QDRANT_PATH); collection enterprise_incidents; cosine distance
Chunking strategy	One vector per incident: 'Service / Title / Root Cause' text - a deliberate choice because incidents are small atomic records; no text chunking is applied
Indexing	scripts/seed.py validates required fields, embeds, upserts with deterministic UUIDv5 ids (safe to re-run), verifies stored count, exits non-zero on failure
Retrieval	Top-k cosine search (limit 5) with optional payload filters (service/severity/status); payload keyword indexes for filtered search
Quality gate	Results below MIN_RELEVANCE_SCORE (0.30) dropped before entering evidence; the strength heuristic additionally requires >= 0.45 for HIGH
Context construction	EvidenceBundle - formatted prompt sections with hard caps: 5 incidents, 5 deployments, 30 log lines, 3000 runbook chars - the raw log file is never dumped into the prompt
Response generation	Strict markdown template parsed back into InvestigationResult (summary, impact, root cause, lists, confidence)
Guardrails	Prompt forbids inventing data; confidence clamped by _final_confidence to the validator's strength; runbooks labelled guidance-only; timeline uses only real timestamps
Fallbacks	Gemini failure after retries - _fallback_result: evidence-only report, no root cause invented, explicit user message (verified live in this analysis)

14.2 Multi-Agent Design

Four specialised retrieval agents (incident, deployment, log, runbook) plus a generation agent are coordinated by the LangGraph workflow. Each agent owns one data source and returns typed evidence, making the pipeline testable in isolation (tests/test_agents.py). This is a fixed orchestration graph, not an autonomous agent loop - deterministic, auditable and cheap.

14.3 MCP

Partially implemented. backend/mcp_server/server.py defines a FastMCP server ('Enterprise RCA Tools') with four working tools that reuse the real service layer: search_incidents_tool, get_deployment, search_logs, get_runbook. It runs standalone (python -m mcp_server.server) and is intentionally not wired into the main API or consumed by any agent yet - the README lists it as future agent-integration surface.

14.4 Prompt Engineering Notes

The RCA prompt embeds the user question, the validator's evidence strength, formatted incident/deployment/log/timeline sections and the runbook, then gives seven explicit instructions (use only the evidence; separate fact from inference; declare inconclusive when strength is LOW/INSUFFICIENT; treat runbooks as guidance; explain why the chosen root cause beats alternatives; state exactly one confidence level with justification and never claim High against low evidence strength). The markdown response is parsed with section extraction (_build_result) so the UI gets structured fields, not raw text.

15. Complete Application Workflow

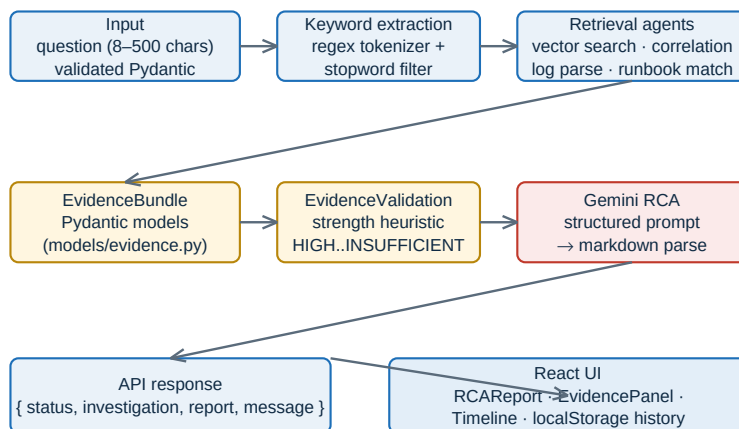
Investigation workflow (happy path): User - React UI (InvestigationInput) - services/api.js investigateStream() - POST /investigate/stream - rate-limiter check - SSE investigation_started - LangGraph: question_analysis (keywords) - incident_retrieval (Qdrant + threshold) - deployment_correlation (deployment_id) - log_retrieval (parse/score/cap) - runbook_retrieval (keyword match) - evidence_aggregation (EvidenceBundle) - evidence_validation (strength + timeline) - rca_generation (Gemini + clamp) - SSE investigation_completed - React renders RCAReport with EvidencePanel - history saved to localStorage.

Each node completion emits an SSE _completed event that the LoadingState checklist marks done - the UI never shows progress the backend has not confirmed.

Failure workflows (implemented):

- Gemini unavailable - retries (3 attempts) then evidence-only fallback result with status 'error' and an explanatory message (verified live in this analysis).
- Any retrieval-node failure - recorded in errors/retrieval_errors; workflow continues; the validator adds a note to the result.
- Rate limit exceeded - HTTP 429 with a friendly message (tested).
- SSE stream failure - investigation_error event with generic message.
- Invalid input - HTTP 422 with generic message (tested: missing, short, blank, overlong questions).

16. Complete Data Flow



Error paths: node failure → recorded in retrieval_errors · Gemini failure → retry ×3 → evidence-only fallback (no invented content)

Figure 4 - End-to-end data flow from input to UI, with error paths

Stage	What Happens	Logging / Errors
Input	Question arrives as JSON; Pydantic enforces 8-500 chars and rejects blank/whitespace	422 validation errors logged with request path
Processing	Keyword extraction (regex + stopword list) feeds retrieval; Qdrant cosine scores; log parser scores and sorts by severity/recency	Per-node info logs with counts; sub-threshold drops logged at debug
Business logic	Evidence aggregated into EvidenceBundle with RetrievalMetadata (counts, min relevance, retrieval_errors); validator computes strength + notes	Evidence strength logged; notes attached to result
External services	Gemini call with per-attempt logging; Langfuse spans with ms timings (also logged locally so latency is observable without Langfuse)	Failures logged with full stack internally only
Output	Structured InvestigationResult + markdown fallback; SSE events; React state; localStorage history	Trace metadata records only question_length - not the question itself - to avoid storing sensitive data
Error paths	Uniform {status, message} envelope; unhandled exceptions caught globally; stack traces never reach the client	logger.exception at the API boundary

17. Project Folder Structure

```
Enterprise-ai-operations-copilot/
|-- README.md Overview, architecture, setup, API, production notes
|-- .env.example / .gitignore Configuration template and VCS exclusions
|-- docs/ Screenshots; report generator
|-- backend/
| |-- app.py FastAPI entry, CORS, error handlers, lifespan
| |-- config.py Environment-driven settings (single source of truth)
| |-- requirements.txt Pinned dependencies
| |-- api/routes.py Endpoints + Pydantic request/response models
| |-- agents/ incident, deployment, log, runbook, gemini_rca
| |-- workflows/rca_graph.py LangGraph 8-node investigation graph
| |-- models/evidence.py Pydantic evidence contracts (cross-stage types)
| |-- vector_store/ Qdrant client/setup/search + seed wrapper
| |-- utils/ data_store (lru_cache), rate_limit
| |-- observability/ Langfuse client + tracer (fail-soft)
| |-- mcp_server/server.py Optional FastMCP tool server (4 tools)
| |-- scripts/seed.py Canonical Qdrant seeding command
| |-- data/ incidents, deployments, logs, runbooks
| |-- tests/ conftest.py, test_agents.py, test_api.py
| -- qdrant_data/ Embedded Qdrant persistence (generated)
-- frontend/
|-- package.json / vite.config.js / eslint.config.js
|-- .env.example VITE_API_URL
-- src/
|-- App.jsx Layout, state, SSE flow, history, theme
|-- services/api.js Backend client (investigate, stream, health)
-- components/ Header, Sidebar, InvestigationInput, InvestigationMessage,
RCAReport, EvidencePanel, Timeline, LoadingState, Feedback
```

Key responsibilities: **config.py** centralises every environment variable so no module reads `os.environ` directly; **models/evidence.py** is the contract layer that makes stages independently testable; **scripts/seed.py** is the single canonical way to populate the vector store (an older entry point, `vector_store/load_enterprise_data.py`, is kept as a thin deprecated wrapper).

18. Module & Implementation Details

Module	Main Logic	Dependencies	I/O
config.py	Settings reading env once (Gemini, Langfuse, Qdrant, CORS, rate limits, validation limits, MIN_RELEVANCE_SCORE, LOG_LEVEL); root logging; warns when GEMINI_API_KEY missing	python-dotenv	settings object

Module	Main Logic	Dependencies	I/O
api/routes.py	4 endpoints; Pydantic models with field_validator blank checks; rate limiter check; tracer per investigation; graph.invoke / graph.stream(updates) - SSE per node; response envelope builder	FastAPI, LangGraph app	HTTP JSON / SSE
workflows/rca_graph.py	TypedDict RCAState; 8 nodes each with try/except/finally tracing; _build_timeline from real timestamps (cap 15); strength heuristic; compiled StateGraph; NODE_STAGE_IDS for SSE	agents, models	final state
agents/incident_agent.py	search_incidents + threshold filter (0.30); returns incidents + scores	vector_store, config	list[IncidentEvidence]
agents/deployment_agent.py	deployment_id exact join; service fallback only if no ID matched (cap 10)	utils.data_store	list[DeploymentEvidence]
agents/log_agent.py	Regex LOG_LINE_RE parse; LEVEL_PRIORITY + ERROR_TERMS scoring; sort by (-relevance, timestamp); cap 40	utils.data_store	list[LogEvidence]
agents/runbook_agent.py	RUNBOOK_KEYWORDS domain map; substring/token scoring; best match wins; content capped 4000 chars	utils.data_store	RunbookEvidence None
agents/gemini_rca_agent.py	RCA_PROMPT template; formatting helpers with caps; 3 attempts; _build_result parses markdown sections; _final_confidence clamp; _fallback_result evidence-only	google-genai, config, tracing	InvestigationResult (never raises)
vector_store/*	qdrant_client (path client); qdrant_setup (ensure_collection, 384-dim cosine, 3 payload indexes); search_service (encode + query_points + filters)	qdrant-client, sentence-transformers	scored points
scripts/seed.py	validate_data (required fields) - ensure_collection - embed - upsert (uuid5 ids) - count verification - exit codes	qdrant, sentence-transformers	seeded collection
utils/rate_limit.py	Sliding-window per-IP limiter using time.monotonic; module-level instance from settings	config	allow/deny
observability/*	Defensive Langfuse init (None on failure); noop-safe spans; stage duration in ms	langfuse SDK	traces
frontend/src/App.jsx	Message array state; runInvestigation prefers stream with fallback to non-streaming; pushHistory with quota fallback; theme + history persistence	components, api.js	UI
frontend/src/services/api.js	fetchWithTimeout (AbortController); SSE parser over ReadableStream; STAGES mirroring NODE_STAGE_IDS; checkHealth	fetch API	promises/events

19. Validation & Error Handling

Layer	Mechanism
API input validation	Pydantic Field constraints (question 8-500, query 2-300, limit 1-20) + field_validator rejecting blank/whitespace; returns 422 with a generic user-safe message while details go to logs
Frontend validation	maxLength 500 on the textarea, non-blank submit guard, live counter; matches backend limits by design
Vector store validation	Seed script requires id/service/severity/status/title/root_cause and aborts on missing fields; verifies stored count after upsert
Workflow exceptions	Each LangGraph node catches its own exceptions, stores empty results, appends the stage to errors - partial evidence still produces a report
API error responses	Dedicated handlers for HTTPException, RequestValidationError and Exception; uniform {status, message}; a unit test asserts internal error text never leaks into the 500 body
External service failures	Gemini: 3 attempts then evidence-only fallback (verified live). Langfuse: every call wrapped, failures logged as warnings

Layer	Mechanism
Retry logic	GEMINI_MAX_RETRIES (default 2, i.e. 3 attempts) with per-attempt logging
Fallbacks	Qdrant collection auto-created at startup if missing; UI falls back from streaming to non-streaming investigate(); localStorage quota fallback stores metadata only
Logging	Root logging configured in config.py; httpx/httpcore/urllib3 noise suppressed; stage timings logged in ms; rate-limit violations warned
Recovery	Stateless API - failed requests can simply be retried; rate-limiter window self-expires; refresh_data_cache() supports hot data reload

20. Security Analysis

20.1 Implemented Security

- **Secrets management:** keys only via environment variables (backend .env, frontend VITE_API_URL); .env files are git-ignored; .env.example contains placeholders only. No API keys, passwords or tokens are hardcoded anywhere in the repository (verified by inspection).
- **CORS:** explicit allow-list from CORS_ORIGINS (default localhost:5173 / 127.0.0.1:5173); methods limited to GET/POST; headers limited to Authorization/Content-Type; no wildcard.
- **Rate limiting:** per-IP sliding window (10/60 s default) on the expensive LLM endpoints to bound API spend.
- **Input validation:** strict Pydantic bounds mirrored on the client; prompt-injection surface reduced by capping and structuring all context fed to Gemini.
- **Error hygiene:** internal exceptions and stack traces are logged, never returned; a unit test explicitly asserts no internal detail leaks in the 500 body.
- **Data minimisation:** Langfuse traces record question length, not the question text.
- **Parsing safety:** unparseable log lines are preserved verbatim rather than interpreted; no eval or dynamic execution exists in the codebase.
- **Injection:** no SQL is used; Qdrant queries use typed filter objects, not string interpolation.

20.2 Recommended Security Improvements

- **Authentication & authorization:** none today - required before any deployment beyond localhost (the README acknowledges this). Recommended: token/OAuth-based auth with protected routes.
- **XSS:** React escapes interpolated content by default and react-markdown is used without custom HTML rendering; still, a strict Content-Security-Policy is recommended for defence in depth.
- **CSRF:** low risk with the current no-cookie design; becomes relevant once cookie sessions are introduced.
- **Rate limiter scope:** in-memory and per-process - a distributed deployment needs a shared store (e.g. Redis) and a defined X-Forwarded-For trust policy.
- **Dependency security:** no automated scanning (pip-audit / npm audit / Dependabot) is configured.
- **Transport security:** no HTTPS/TLS configuration - required for any hosted deployment.

21. Testing

The project ships a pytest suite (backend/tests) with external services (Qdrant, Gemini, LangGraph) mocked, so no API credits are consumed and no network is required. The suite was executed as part of this report's verification.

File	Scope	Notable Assertions
tests/conftest.py	Makes the backend package importable from any working directory; quiets logging	-

File	Scope	Notable Assertions
tests/test_api.py (11 tests)	Health endpoint; /investigate validation (missing/short/blank/overlong question); structured payload shape; validation block; no raw exception leakage on workflow failure; rate limiting (3rd request - 429); /search validation and limit bounds	Error messages are user-safe; markdown fallback present; rate limit enforced
tests/test_agents.py (15 tests)	Deployment correlation by id and service fallback; log parsing/prioritisation/keyword fallback; runbook matching (payroll, jwt, unrelated); timeline uses only real timestamps; evidence layer authority over confidence; Gemini success parse and failure fallback	No invented root cause on failure; confidence clamped; chronological timeline

Actual result (run for this report): 26 passed, 2 warnings in 28.83 s (pytest 8.3.4, Python 3.12.10). The two warnings are third-party deprecations inside langgraph and starlette's test client - not project issues. No frontend unit/E2E tests exist (npm scripts provide dev, build, lint, preview only).

22. Verification & Quality Assurance

Every row below was executed during the preparation of this report on the actual workspace.

Verification	Command / Method	Result	Status
Backend test suite	python -m pytest tests/ -v (backend/)	26 passed, 2 warnings in 28.83 s	Pass
Frontend production build	npm run build (frontend/)	201 modules transformed; dist/ generated; JS 337.10 kB (gzip 104.72 kB); built in 1.00 s; exit code 0	Pass
Backend startup	python -m uvicorn app:app --port 8017	Uvicorn running; lifespan warmed cache; Qdrant collection validated	Pass
Health endpoint	GET http://127.0.0.1:8017/health	status=healthy, qdrant=true, gemini_configured=false	Pass
Semantic search endpoint	POST /search {query: 'payroll failing', limit: 3}	status=success, 3 scored results returned	Pass
Investigation without Gemini key	POST /investigate (payroll question)	Graceful fallback: status=error, confidence LOW clamped while evidence_strength HIGH preserved, message 'AI analysis is temporarily unavailable...' - observed evidence still returned	Pass (fallback path)
Log evidence prioritisation	agents/log_agent.py + unit tests	ERROR/WARN prioritised, 40-line cap	Pass
Secrets scan	Inspection of .env files, .gitignore, source	frontend/.env contains only VITE_API_URL; backend .env absent; .env.example placeholders only	Pass
Lint	npm run lint	ESLint 9 flat-config fails to load under the installed Node 24 environment (config-loader error)	Fail - see limitations
Seed re-run	python -m scripts.seed (documented command)	Not re-run in this analysis; idempotent design verified by code review (uuid5 ids, count check)	Not re-run
Gemini generation with a real key	requires GEMINI_API_KEY	Key not present in this workspace - generation path verified via mocks and the fallback path live	Not verified live

23. Performance Analysis

No formal benchmark suite exists in the repository; the observations below come from the live run performed for this report plus design-level analysis. Per-request timings were not captured into a benchmark artefact and are

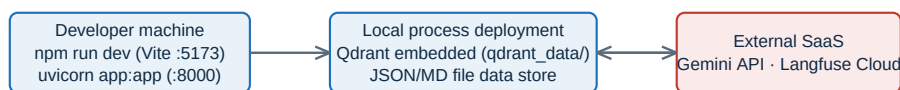
therefore not fabricated.

Aspect	Observation / Design Evidence
Frontend bundle	337.10 kB JS (104.72 kB gzip) + 18.48 kB CSS (4.32 kB gzip) - small single-page bundle; 1.00 s production build
Static data access	lru_cache(1) means deployments/logs/runbooks are read from disk once per process, not per request
Embedding cost	SentenceTransformer instantiated once at import; per-query encoding is a single 384-dim inference (tens of milliseconds on CPU for short queries)
Vector search	Embedded Qdrant over 20 points with payload indexes - very fast at this scale; designed to scale with server mode
LLM latency	Dominant cost: gemini-2.5-flash call, timeout 60 s; the UI allows 180 s. Stage timings are logged in ms by InvestigationTracer (and visible in Langfuse when configured)
Streaming	SSE keeps the user informed during long generations instead of a blocking spinner
Memory	Embedding model + 550 log lines + JSON data are small; the in-memory rate limiter grows with distinct client IPs (unbounded - see Section 24)
Large-data behaviour	Evidence caps bound prompt size regardless of data growth: 5 incidents, 5 deployments, 40 log lines, 4000 runbook chars

24. Scalability

Dimension	Current Behaviour	Bottleneck / Recommendation
Users increase	Single uvicorn process; stateless API	Run multiple workers/instances; move the rate limiter to Redis
Data increase	Embedded Qdrant file-backed collection; lru_cache assumes data fits in memory	Switch to Qdrant server mode (QDRANT_URL); move data files to a database; add incremental indexing
API traffic	FastAPI + Pydantic are efficient; the heavy work is the LLM call	Queue investigations (e.g. Celery/RQ) instead of holding HTTP connections; horizontal scaling behind a load balancer
Concurrent requests	SSE holds one connection per active investigation	Worker/threadpool sizing; consider WebSockets with backpressure at larger scale
AI usage increase	Rate limit (10/60 s per IP) bounds per-client spend but not global spend	Add a global budget/quota, response caching for repeated questions, and a cheaper model tier for triage

25. Deployment & DevOps



Not identified in the current implementation: Dockerfile, CI/CD pipeline, cloud hosting, monitoring/alerting, production database.

Figure 5 - Current deployment topology (local process, external SaaS LLM/observability)

Concern	Status in This Project
Build	Frontend: npm run build (Vite, verified). Backend: none needed (interpreted Python)
Environment configuration	.env files per tier; .env.example templates; config.py resolves repo-root or backend .env automatically
Docker	Not identified in the current implementation (no Dockerfile/compose)
CI/CD	Not identified in the current implementation (no pipeline config in the repository)
Hosting platform	Local machine only (uvicorn + Vite); no cloud deployment configured

Concern	Status in This Project
Database deployment	Embedded Qdrant directory backend/qdrant_data; no managed database
Environment variables	Backend: GEMINI_API_KEY (required), LANGFUSE_*, CORS_ORIGINS, GEMINI_MODEL/TIMEOUT_MS/MAX_RETRIES, QDRANT_PATH/COLLECTION, INVESTIGATE_RATE_*, QUESTION_MIN/MAX_LENGTH, MIN_RELEVANCE_SCORE, LOG_LEVEL. Frontend: VITE_API_URL
Monitoring	Structured Python logging + optional Langfuse tracing; GET /health for probes; no external monitoring/alerting
Production configuration guidance	README documents: real CORS origins, auth requirement, Qdrant server mode, process manager, LOG_LEVEL=INFO

26. Installation & Setup

The steps below are taken from the project README and verified against the codebase.

Prerequisites

- Python 3.12 (verified on 3.12.10), Node.js 18+ (verified on v24.19.0), npm.
- A Google Gemini API key (GEMINI_API_KEY) for LLM generation; Langfuse keys optional.

Backend

```
cd backend
pip install -r requirements.txt
copy ../.env.example .env # then set GEMINI_API_KEY
python -m scripts.seed # validate, embed, upsert incidents into Qdrant
uvicorn app:app --reload # http://127.0.0.1:8000 - Swagger at /docs
```

Frontend

```
cd frontend
npm install
copy .env.example .env # VITE_API_URL=http://127.0.0.1:8000
npm run dev # http://localhost:5173
```

Tests and production build

```
cd backend && python -m pytest tests/ -v # 26 tests, external services mocked
cd frontend && npm run build # outputs dist/
```

Database setup consists solely of the seed step above - the Qdrant collection is created automatically at app startup if missing, and the seed script is idempotent (deterministic UUIDv5 point ids).

27. Problems Found & Improvements Made

Git history shows a single initial commit (v1.0.0) with substantial uncommitted restructuring: early experimental modules (backend/rag/*, backend/embeddings/*, ad-hoc test scripts) were deleted and replaced by the current structured architecture (config.py, models/, utils/, observability/, tests/). Because history cannot establish before/after behaviour, items below are documented as engineering improvements visible in the current design, or as potential issues identified during analysis - not as verified historical fixes.

Problem / Improvement	Cause	Solution (as implemented)	Result
Unstructured dict passing between pipeline stages (removed design)	Early RAG modules passed arbitrary dictionaries	Typed Pydantic evidence layer (models/evidence.py) is now the contract between agents, validation, LLM and API	Design improvement; each agent unit-testable in isolation

Problem / Improvement	Cause	Solution (as implemented)	Result
Hallucination risk in LLM RCAs	LLMs can claim high confidence without evidence	Deterministic evidence-validation node + <code>_final_confidence</code> clamping + strict prompt rules	Verified by unit tests (authority test; failure test asserts no invented root cause)
Raw exceptions leaking to users	Typical default FastAPI behaviour	Global handlers normalise every error into <code>{status, message}</code> ; details only in logs	Verified: <code>test_no_raw_exception_on_workflow_failure</code> passes
Unbounded LLM spend	Expensive Gemini calls on a public endpoint	Per-IP sliding-window rate limiter (default 10/60 s)	Verified: <code>test_rate_limiting</code> passes (3rd request - 429)
Silent observability failures	Optional Langfuse could break the pipeline	Defensive client init + noop spans; every tracing call wrapped	Design-level; app ran here with Langfuse unconfigured and no errors
Potential issue: unbounded in-memory rate-limiter map	RateLimiter keeps every client IP forever	None yet - add pruning or an LRU bound	Open
Potential issue: lint script broken in current environment	ESLint 9 flat-config fails to load under the installed Node toolchain	None yet - pin a compatible ESLint/Node pair or update the config	Open (verified failing)
Potential issue: non-ASCII mojibake in a few UI literals	Windows encoding of special characters in JSX strings	None yet - normalise to ASCII or enforce UTF-8 toolchain-wide	Open (cosmetic)

28. Current Strengths

- **Architecture:** clean layering (API - workflow - agents - data) with typed Pydantic contracts; a deterministic 8-node graph instead of a free-form agent loop makes behaviour predictable and testable.
- **Honesty-by-design:** the strongest engineering decision in the project - evidence strength gates LLM confidence, timelines use only real timestamps, runbooks are quarantined as guidance, and the fallback path returns evidence without AI inference.
- **Operational resilience:** every external dependency (Gemini, Langfuse, Qdrant) fails soft or falls back; nodes are individually fault-isolated.
- **Testability:** 26 fast, credit-free tests pinning down the most safety-critical behaviours (confidence clamping, error envelopes, rate limiting, timeline integrity).
- **Code organisation:** single config source; pinned dependencies; intent-documenting docstrings; deprecated entry points kept as thin wrappers rather than dead code.
- **UI/UX:** chat-style experience with live pipeline progress, inspectable evidence accordions with source labels, searchable history, theming and accessible controls - unusually mature for a prototype.
- **Security basics:** strict CORS, rate limiting, secrets discipline and user-safe errors done correctly from the start.

- **Documentation:** the README accurately describes architecture, endpoints, environment variables, tests and production considerations - it matches the code (verified during this analysis).

29. Known Limitations

Severity	Limitation
Critical (for public use)	No authentication or authorization - anyone who can reach the API can invoke Gemini-backed investigations and consume the API key's quota
High	No server-side persistence: history, feedback and multi-user state live only in the browser; the data set is a fixed sample, not a live telemetry feed
High	Single-process constraints: embedded Qdrant and in-memory rate limiter prevent horizontal scaling
Medium	No Docker/CI-CD, HTTPS, or monitoring/alerting; the deployment story is local-only
Medium	Runbook and log retrieval are keyword heuristics (not semantic) - recall depends on curated keyword maps; question analysis is a stopword tokenizer, not an NLP model
Medium	npm run lint fails in the current environment (ESLint 9 flat-config load error under Node 24)
Low	Minor non-ASCII mojibake in a few UI literals; version inconsistency (Sidebar shows v1.2.0, package.json/app are 1.1.0)
Low	Feedback component stores locally only; App passes a no-op onFeedback callback

30. Recommended Improvements

30.1 High Priority

- **Add authentication and authorization** (JWT or OAuth via FastAPI dependencies) before any non-local deployment. Why: without it, anyone can spend the Gemini quota and read enterprise incident data.
- **Harden rate limiting and add a global spend cap** (Redis-backed limiter + daily LLM budget). Why: the in-memory limiter does not survive restarts or multiple workers, and a per-IP limit does not bound total cost.
- **Fix the lint toolchain** (pin compatible ESLint/Node versions or migrate the flat config). Why: CI cannot enforce code quality until lint runs.

30.2 Medium Priority

- **Introduce server-side persistence** (PostgreSQL/Supabase for users, investigations, feedback). Why: replaces localStorage-only history and enables multi-user analytics (the README already plans this).
- **Dockerise backend and frontend** with a compose file including a Qdrant server. Why: reproducible environments and a path to cloud deployment.
- **Add a CI pipeline** (GitHub Actions: pytest + npm build + lint). Why: keeps the 26-test suite meaningful on every commit.
- **Upgrade retrieval quality:** embed runbooks for semantic matching; consider hybrid (vector + keyword) log retrieval. Why: removes dependence on hand-maintained keyword maps.
- **Wire up or remove the MCP server.** Why: it currently duplicates access logic with no consumer.
- **Unify version strings** (1.1.0 vs Sidebar v1.2.0). Why: confusing for users and support.

30.3 Low Priority

- Cache Gemini responses for identical questions (dedup + cost saving).
- Stream the markdown report at token level to further reduce perceived latency.
- Add structured JSON logging with request-ID correlation across stages.
- Deepen accessibility (focus trapping in the sidebar overlay, keyboard navigation for accordions).
- Normalise non-ASCII UI strings to avoid mojibake across encodings.

31. Future Enhancements

Realistic extensions that fit the existing architecture (all are recommendations, not implemented features):

- **Live data ingestion:** connectors to ticketing (Jira/ServiceNow), APM (Datadog, Prometheus) and log platforms feeding the same evidence models.
- **Multi-user RCA workspace:** per-user investigation threads, sharing, and RCA review/approval workflows on top of the planned persistence layer.
- **Agent-mode investigations:** use the existing FastMCP tools with an agentic loop (function calling) for multi-step, exploratory RCAs.
- **Feedback-driven improvement loop:** export the already-captured feedback to Langfuse datasets and evaluate prompt variants against them.
- **Deployment-risk scoring:** correlate historical incident rates per service/deployment to flag risky changes.
- **Postgres pgvector option:** consolidate vectors and relational data once the database layer exists.
- **Notification integrations:** push finished RCAs to Slack/Teams webhooks.

32. Maintenance & Support

Area	Guidance
Configuration	All runtime settings live in backend/config.py sourced from environment variables (see Appendix); the frontend has a single variable, VITE_API_URL
Important modules	workflows/rca_graph.py (pipeline), agents/* (retrieval and generation), vector_store/* (search), utils/rate_limit.py (abuse protection), observability/* (tracing)
Common tasks	Add an incident: append to data/incidents.json and re-run python -m scripts.seed. Add a runbook: drop a markdown file into data/runbooks/ and add its domain keywords to RUNBOOK_KEYWORDS. Clear data caches: utils.data_store.refresh_data_cache()
Vector store maintenance	Embedded Qdrant data lives in backend/qdrant_data - back it up or re-seed deterministically; verify counts via the seed script's built-in check
Deployment maintenance	Run uvicorn under a process manager; LOG_LEVEL=INFO in production; rotate the Gemini key via environment only
Logging	Root logger configured in config.py; stage timings logged in ms - watch 'Stage ... completed in ... ms' lines for latency regressions
Troubleshooting	qdrant=false in /health - check qdrant_data or re-run the seed. gemini_configured=false - set GEMINI_API_KEY. HTTP 429 - rate limit reached. RCA with a notice banner - Gemini failed and the evidence-only fallback was returned
Special attention	Never log or commit secrets; keep evidence caps aligned when changing prompt size; keep the frontend STAGES list in sync with backend NODE_STAGE_IDS

33. GitHub / Repository Readiness

Item	Assessment
README	Excellent: architecture diagram, tech stack, structure, setup, environment variables, API table, test command, production considerations, future evolution
.gitignore	Present at root and in frontend; node_modules, dist, .env and caches excluded
Dependency files	requirements.txt fully pinned; package-lock.json committed
Secrets	No secrets in the repository; .env.example placeholders only; frontend/.env holds a non-secret URL and is local-only
Tests	26 backend tests included and passing; no frontend tests
Scripts	seed.py gives reproducible vector-store setup; npm scripts are standard
Generated files	qdrant_data/, dist/, __pycache__ exist locally (git-ignored); stray logs (uv_*.log, npm_install*.log) in the parent folder - move or delete before publishing

Item	Assessment
Reproducibility	High: pinned dependencies + deterministic seeding + mocked tests; a fresh clone reproduces behaviour given a Gemini key
Overall readiness	Good for a portfolio repository after removing local logs/artifacts and fixing the lint script; production readiness requires auth, Docker and CI (Section 30)

34. Resume / Portfolio Value

Skill	Evidence in the Project
Full-stack development	Complete React 19 SPA + FastAPI service delivering one product end to end
AI / RAG engineering	Qdrant vector store, bge-small embeddings, relevance gating, grounded Gemini prompting, confidence clamping, evidence-only fallback
Multi-agent & workflow orchestration	LangGraph StateGraph with typed state, fault-isolated nodes and stage-level observability
API development	FastAPI with Pydantic contracts, SSE streaming, uniform error envelopes, Swagger docs, rate limiting
Python backend engineering	Modular package design, lru_cache data caching, defensive error handling, configuration discipline
React / modern frontend	Hooks-based architecture, SSE parsing over fetch streams, accessible components, theming
Observability	Optional Langfuse tracing with fail-soft design and per-stage timing instrumentation
Testing / QA	26-test pytest suite mocking external services; assertion of security-relevant behaviours (error leakage, rate limiting)
Security fundamentals	CORS allow-listing, secrets management, input validation, data minimisation in traces
Data engineering (light)	Deterministic idempotent seeding pipeline with validation and verification
Technical documentation	Accurate README plus this evidence-based technical report

35. Final Technical Summary

Dimension	Summary
Architecture	React 19 SPA - FastAPI - LangGraph 8-node pipeline - Qdrant (embedded) + file data store - Gemini LLM; SSE progress streaming; fail-soft optional Langfuse tracing
Technology	Python 3.12 / FastAPI 0.115 / LangGraph 0.2 / qdrant-client 1.12 / sentence-transformers 3.3 / google-genai 0.6 / React 19 / Vite 6 (all pinned)
Features	Grounded RCA generation, RAG incident retrieval with thresholding, deployment correlation, log/runbook evidence, evidence validation, streaming UI, history, feedback, health monitoring
Data flow	question - keywords - agents - EvidenceBundle - EvidenceValidation - Gemini - InvestigationResult - SSE/JSON - React UI - localStorage; uniform error envelope everywhere
Security	Implemented: CORS allow-list, rate limiting, secrets hygiene, input validation, user-safe errors, trace minimisation. Missing: authentication, HTTPS, dependency scanning
Testing	26/26 pytest tests passing in 28.83 s (services mocked); frontend verified via production build; no frontend test suite
Performance	Small bundle (337 kB / 105 kB gzip); cached data; bounded prompts; LLM call is the dominant latency and is instrumented per stage
Deployment	Local-process only today; Docker/CI/cloud documented as future work; idempotent seeding and health endpoint ready for orchestration
Current status	Functional, tested prototype (v1.1.0) - strong engineering foundations; authentication and persistence are the gating items for production

36. Appendix

A. API Summary

GET /health - status, qdrant, gemini_configured. POST /search - semantic incident search (query 2-300, limit 1-20). POST /investigate - synchronous RCA. POST /investigate/stream - SSE RCA with per-stage events. All errors: {"status": "error", "message"}. Rate limit: 10/60 s per IP on both /investigate endpoints.

B. Data Summary

Qdrant collection enterprise_incidents (384-dim cosine, payload indexes service/severity/status) seeded from data/incidents.json (20 incidents, required fields id/service/severity/status/title/root_cause). data/deployments.json: 20 deployments joined by deployment_id. data/logs.txt: 550 lines parsed by regex. data/runbooks: 5 markdown runbooks (database_timeout, email_delivery_failure, gateway_outage, jwt_auth_failure, payment_failure).

C. Environment Variable Names (values never documented)

Backend: GEMINI_API_KEY, GEMINI_MODEL, GEMINI_TIMEOUT_MS, GEMINI_MAX_RETRIES, LANGFUSE_PUBLIC_KEY, LANGFUSE_SECRET_KEY, LANGFUSE_HOST, CORS_ORIGINS, QDRANT_PATH, QDRANT_COLLECTION, INVESTIGATE_RATE_LIMIT, INVESTIGATE_RATE_WINDOW_SECONDS, QUESTION_MIN_LENGTH, QUESTION_MAX_LENGTH, MIN_RELEVANCE_SCORE, LOG_LEVEL. Frontend: VITE_API_URL.

D. Key Configuration Defaults

Setting	Default
GEMINI_MODEL	gemini-2.5-flash
GEMINI_TIMEOUT_MS / MAX_RETRIES	60000 / 2 (3 attempts)
CORS_ORIGINS	http://localhost:5173, http://127.0.0.1:5173
INVESTIGATE_RATE_LIMIT / WINDOW	10 / 60 s
QUESTION_MIN / MAX_LENGTH	8 / 500
MIN_RELEVANCE_SCORE	0.30 (HIGH requires 0.45)
QDRANT_PATH / COLLECTION	backend/qdrant_data / enterprise_incidents
Evidence caps	5 incidents, 5 deployments, 40 log lines (30 to prompt), 3000-4000 runbook chars, 15 timeline events

E. Glossary

- **RCA** - Root Cause Analysis: structured investigation of why an incident happened.
- **RAG** - Retrieval-Augmented Generation: grounding an LLM in retrieved documents.
- **Evidence strength** - validator-computed quality level (HIGH/MEDIUM/LOW/INSUFFICIENT) of retrieved evidence.
- **SSE** - Server-Sent Events: one-way HTTP streaming used for live pipeline progress.
- **MCP** - Model Context Protocol: standard protocol for exposing tools to LLM agents (FastMCP server here).
- **Embedded Qdrant** - Qdrant running in-process against a local file path instead of a server.
- **Langfuse** - LLM observability platform for traces, spans and evaluations.

F. Verification Information

This report was generated from a static analysis of the full codebase plus live verification on September 4, 2026: pytest run (26/26 passed), npm production build (successful), backend startup and health/search/investigate endpoint probes (all behaved as designed, including the Gemini-fallback path). No secrets were collected or reproduced. Sections marked 'Not identified in the current implementation' or 'Not verified live' reflect genuine gaps rather than assumptions.